

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-213Б-23

Студент: Пономарев А.А

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 08.01.25

Москва, 2024

Постановка задачи

Вариант 4.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Отсортировать массив целых чисел при помощи TimSort

Общий метод и алгоритм решения

Использованные системные вызовы:

- `int write(int fd, void* buf, size_t count);` – записывает `count` байт из `buf` в `fd`.
- `int pthread_create(pthread_t *restrict thread, const pthread_attr_t *restrict attr, void *(*start_routine)(void *), void *restrict arg);` – создает новый поток выполняющий функцию `start_routine` с аргументом `restrict arg`, и записывает Id созданного потока в `thread`. Вызов возвращает 0, если успешен иначе код ошибки.
- `int pthread_join(pthread_t thread, void **retval);` – ожидает завершения потока с `id thread`, и если `retval` не равен `NULL`, записывает результат работы функции потока в `retval`. Вызов возвращает 0, если успешен иначе код ошибки

Метод решения:

Деление задачи на части: Код использует подход с многозадачностью, где данные разделяются на несколько частей (в зависимости от количества потоков), и каждая часть обрабатывается отдельно разными потоками.

Сортировка данных с помощью Timsort: Для каждой части массива используется сортировка Timsort. Этот алгоритм сочетает два подхода: сортировку вставками (для малых подмассивов) и сортировку слиянием (для больших массивов). Timsort — это оптимизированная версия сортировки слиянием, которая обеспечивает высокую производительность на реальных данных.

Параллельная обработка с использованием потоков: Каждый поток сортирует свою часть массива параллельно с другими потоками, что позволяет ускорить процесс сортировки.

Слияние отсортированных частей: После того как все потоки завершат свою работу, отсортированные части массива сливаются в один отсортированный массив. Этот этап выполняется с использованием стандартной сортировки слиянием.

1. Алгоритм:

1. **Инициализация:**
2. Входные параметры: размер массива и количество потоков.
3. Проверка корректности входных данных (например, количество потоков не должно превышать 16).
4. Создание и заполнение массива случайными числами.
5. **Сортировка с использованием потоков:**
6. Разбиение массива на сегменты, каждый из которых будет обрабатываться отдельным потоком.
7. Для каждого потока создается структура `TaskInfo`, в которой указаны индексы начала и конца сегмента данных.
8. Запуск потоков, где каждый поток выполняет сортировку части массива с использованием `Timsort`.
9. После завершения всех потоков выполнение основного потока синхронизируется с помощью `pthread_join`.

Слияние отсортированных сегментов:

10. Основной поток выполняет слияние отсортированных сегментов. Вначале он берет отсортированный массив от первого потока и поочередно сливает его с результатами от последующих потоков.
11. Для слияния используется алгоритм сортировки слиянием, который соединяет два отсортированных массива в один.
12. **Оценка времени работы:**
13. Замер времени до и после выполнения сортировки, чтобы оценить производительность программы.
14. **Проверка корректности:**
15. После сортировки проверяется, отсортирован ли массив, и выводится результат о том, было ли выполнено сортирование правильно.

2. Сложность:

1. **Время сортировки:** основная сложность программы определяется временем работы сортировки `Timsort`, которая имеет сложность $O(n \log n)$ для сортировки всего массива. Разбиение массива на части и использование многозадачности помогает улучшить время работы за счет параллелизации задачи.
2. **Оверхед многозадачности:** С увеличением числа потоков увеличиваются накладные расходы на управление потоками (например, создание потоков, синхронизация, переключение контекста), что может привести к уменьшению производительности при слишком большом количестве потоков.

3. Основные шаги алгоритма:

1. Разбиваем массив на части в зависимости от числа потоков.
2. Каждый поток сортирует свою часть массива с использованием `Timsort`.

3. После завершения сортировки потоков сливаем все отсортированные части в один отсортированный массив.
4. Выводим время выполнения сортировки и проверяем результат.

Этот метод эффективно использует параллельные вычисления для ускорения сортировки на многозадачных системах, однако его производительность зависит от оптимального распределения числа потоков в зависимости от доступных ядер процессора.

Код программы

main.c

```
#include <stdlib.h>
#include <time.h>
#include <stdint.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

const int RUN = 32;

typedef struct TaskInfo {
    int start;
    int end;
    int is_active;
    int* data;
} TaskInfo;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void log_message(int fd, const char* msg) {
    size_t length = 0;
    while (msg[length] != '\0') {
        length++;
    }
    write(fd, msg, length);
}

void log_number(long number) {
    char buffer[64];
    int pos = 0;

    long temp = number;
    if (temp == 0) {
        buffer[pos++] = '0';
    } else {
        while (temp > 0) {
            buffer[pos++] = (temp % 10) + '0';
            temp /= 10;
        }
    }
}
```

```

    for (int i = 0; i < pos / 2; i++) {
        char tmp = buffer[i];
        buffer[i] = buffer[pos - 1 - i];
        buffer[pos - 1 - i] = tmp;
    }
    write(STDOUT_FILENO, buffer, pos);
}

void write_configuration(int array_size, int thread_count) {
    // "Array size: "
    log_message(STDOUT_FILENO, "Array size: ");
    log_number(array_size);
    log_message(STDOUT_FILENO, "\n");

    log_message(STDOUT_FILENO, "Thread count: ");
    log_number(thread_count);
    log_message(STDOUT_FILENO, "\n");
}

void insertion_sort(int* data, int start, int end) {
    for (int i = start + 1; i <= end; i++) {
        int key = data[i];
        int j = i - 1;
        while (j >= start && data[j] > key) {
            data[j + 1] = data[j];
            j--;
        }
        data[j + 1] = key;
    }
}

void merge_sections(int* data, int start, int middle, int end) {
    int left_size = middle - start + 1;
    int right_size = end - middle;

    int* left_part = (int*)malloc(left_size * sizeof(int));
    int* right_part = (int*)malloc(right_size * sizeof(int));

    if (!left_part || !right_part) {
        log_message(STDOUT_FILENO, "Memory error\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < left_size; i++) {
        left_part[i] = data[start + i];
    }
    for (int i = 0; i < right_size; i++) {
        right_part[i] = data[middle + 1 + i];
    }

    int i = 0, j = 0, k = start;

    while (i < left_size && j < right_size) {
        if (left_part[i] <= right_part[j]) {
            data[k] = left_part[i];

```

```

        ++i;
    } else {
        data[k] = right_part[j];
        ++j;
    }
    ++k;
}

while (i < left_size) {
    data[k] = left_part[i];
    ++k;
    ++i;
}

while (j < right_size) {
    data[k] = right_part[j];
    ++k;
    ++j;
}

free(left_part);
free(right_part);
}

void tim_sort(int* array, int left, int right) {
    if (right - left + 1 <= RUN) {
        insertion_sort(array, left, right);
        return;
    }

    int mid = left + (right - left) / 2;
    tim_sort(array, left, mid);
    tim_sort(array, mid + 1, right);
    merge_sections(array, left, mid, right);
}

void* thread_sort_task(void* arg) {
    TaskInfo* data = (TaskInfo*)arg;
    tim_sort(data->data, data->start, data->end);
    pthread_exit(NULL);
}

long get_current_time_ms() {
    struct timeval time_now;
    gettimeofday(&time_now, NULL);
    return time_now.tv_sec * 1000 + time_now.tv_usec / 1000;
}

int main(int argc, char** argv) {
    const int array_size = 1e8;
    int thread_count;

    if (argc != 2) {
        log_message(STDOUT_FILENO, "Usage: program
<thread_count>\n");
        exit(EXIT_FAILURE);
    }

```

```

}

thread_count = atoi(argv[1]);

if (thread_count > 16 || thread_count <= 0) {
    log_message(STDOUT_FILENO, "Wrong number of threads\n");
    exit(EXIT_FAILURE);
}

write_configuration(array_size, thread_count);

int* data = (int*)malloc(sizeof(int) * array_size);

srand((unsigned)time(NULL));
for (int i = 0; i < array_size; i++) {
    data[i] = rand();
}

log_message(STDOUT_FILENO, "Array randomized.\n");

pthread_t* thread_pool = (pthread_t*)malloc(sizeof(pthread_t) *
thread_count);
TaskInfo* tasks = (TaskInfo*)malloc(sizeof(TaskInfo) *
thread_count);

int segment_length = array_size / thread_count;
int offset = 0;

long start_time = get_current_time_ms();

for (int i = 0; i < thread_count; i++, offset += segment_length)
{
    TaskInfo* task = &tasks[i];
    task->data = data;
    task->is_active = 1;
    task->start = offset;
    task->end = (i == thread_count - 1) ? array_size - 1 : offset
+ segment_length - 1;

    pthread_create(&thread_pool[i], NULL, thread_sort_task,
task);
}

for (int i = 0; i < thread_count; i++) {
    pthread_join(thread_pool[i], NULL);
}

pthread_mutex_lock(&lock);
TaskInfo* main_task = &tasks[0];
for (int i = 1; i < thread_count; i++) {
    TaskInfo* next_task = &tasks[i];
    merge_sections(main_task->data, main_task->start, next_task-
>start - 1, next_task->end);
}
pthread_mutex_unlock(&lock);

```

```

long end_time = get_current_time_ms();

log_message(STDOUT_FILENO, "Sorting completed in ");
log_number(end_time - start_time);
log_message(STDOUT_FILENO, " ms\n");

for (int i = 1; i < array_size; i++) {
    if (data[i] < data[i - 1]) {
        log_message(STDOUT_FILENO, "Not sorted\n");
        break;
    }
}

free(tasks);
free(thread_pool);
free(data);

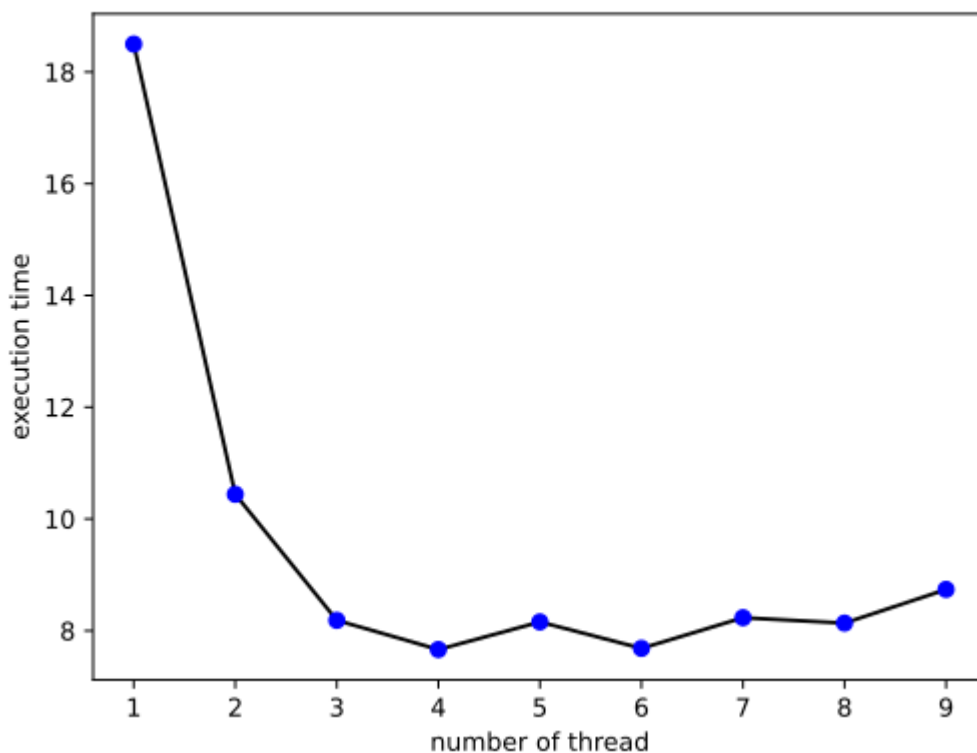
pthread_mutex_destroy(&lock);

return 0;
}

```

Протокол работы программы

Тестирование:



Время выполнения программы для сортировки данных на нескольких потоках сначала уменьшается с увеличением числа потоков, поскольку нагрузка распределяется между ядрами процессора, что ускоряет обработку. Однако после достижения оптимального числа потоков (обычно равного числу

доступных ядер) производительность начинает снижаться из-за накладных расходов на управление потоками, переключение контекста и конкуренцию за ресурсы, такие как память и кэш. Это приводит к тому, что увеличение числа потоков сверх оптимального количества вызывает замедление работы программы

Strace:

```
strace ./main 2

execve("./main", [ "./main", "2"], 0x7ffdd4c868c8 /* 49 vars */) = 0

brk(NULL)                               = 0x555631224000

arch_prctl(0x3001 /* ARCH_??? */, 0x7ffe7fe27470) = -1 EINVAL (Недопустимый аргумент)

access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (Нет такого файла или каталога)

openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3

fstat(3, {st_mode=S_IFREG|0644, st_size=70654, ...}) = 0

mmap(NULL, 70654, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fa4b69f6000

close(3)                                = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpthread.so.0", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0220q\0\0\0\0\0"... , 832) = 832

pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\232e\273F\236E\241\306\373\317\372\345\270*\^327"... , 68,
824) = 68

fstat(3, {st_mode=S_IFREG|0755, st_size=157224, ...}) = 0

mmap(NULL, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa4b69f4000

pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\232e\273F\236E\241\306\373\317\372\345\270*\^327"... , 68,
824) = 68

mmap(NULL, 140408, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7fa4b69d1000

mmap(0x7fa4b69d7000, 69632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x7fa4b69d7000

mmap(0x7fa4b69e8000, 24576, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x17000) = 0x7fa4b69e8000

mmap(0x7fa4b69ee000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c000) = 0x7fa4b69ee000

mmap(0x7fa4b69f0000, 13432, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fa4b69f0000
```

```

close(3) = 0

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3

read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0\300A\2\0\0\0\0"..., 832) = 832

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) =
784

pread64(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0", 32, 848) = 32

pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\7\2C\n\357_\243\335\2449\206V>\237\374\304"..., 68, 880) = 68

fstat(3, {st_mode=S_IFREG|0755, st_size=2029592, ...}) = 0

pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) =
784

pread64(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0", 32, 848) = 32

pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\7\2C\n\357_\243\335\2449\206V>\237\374\304"..., 68, 880) = 68

mmap(NULL, 2037344, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7fa4b67df000

mmap(0x7fa4b6801000, 1540096, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x22000) = 0x7fa4b6801000

mmap(0x7fa4b6979000, 319488, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x19a000) = 0x7fa4b6979000

mmap(0x7fa4b69c7000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e7000) = 0x7fa4b69c7000

mmap(0x7fa4b69cd000, 13920, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fa4b69cd000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa4b67dc000

arch_prctl(ARCH_SET_FS, 0x7fa4b67dc740) = 0

mprotect(0x7fa4b69c7000, 16384, PROT_READ) = 0

mprotect(0x7fa4b69ee000, 4096, PROT_READ) = 0

mprotect(0x5555fcd68000, 4096, PROT_READ) = 0

mprotect(0x7fa4b6a35000, 4096, PROT_READ) = 0

munmap(0x7fa4b69f6000, 70654) = 0

set_tid_address(0x7fa4b67dca10) = 3735

set_robust_list(0x7fa4b67dca20, 24) = 0

rt_sigaction(SIGRTMIN, {sa_handler=0x7fa4b69d7bf0, sa_mask=[],
sa_flags=SA_RESTORER|SA_SIGINFO, sa_restorer=0x7fa4b69e5420}, NULL, 8) = 0

```

```

    rt_sigaction(SIGRT_1, {sa_handler=0x7fa4b69d7c90, sa_mask=[],
sa_flags=SA_RESTORER|SA_RESTART|SA_SIGINFO, sa_restorer=0x7fa4b69e5420}, NULL,
8) = 0

    rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0

    prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
rlim_max=RLIM64_INFINITY}) = 0

    write(1, "Array size: ", 12Array size: )      = 12
    write(1, "1000000000", 9100000000)          = 9
    write(1, "\n", 1
)          = 1
    write(1, "Thread count: ", 14Thread count: )  = 14
    write(1, "2", 12)                            = 1
    write(1, "\n", 1
)          = 1
    brk(NULL)                                    = 0x555631224000
    brk(0x555631245000)                         = 0x555631245000
    mmap(NULL, 400003072, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa49ea63000
    write(1, "Array randomized.\n", 18Array randomized.
)  = 18
    mmap(NULL, 8392704, PROT_NONE,
MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7fa49e262000
    mprotect(0x7fa49e263000, 8388608, PROT_READ|PROT_WRITE) = 0

    clone(child_stack=0x7fa49ea61fb0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|C
LONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEA
RTID, parent_tid=[3736], tls=0x7fa49ea62700, child_tidptr=0x7fa49ea629d0) = 3736
    mmap(NULL, 8392704, PROT_NONE,
MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7fa49da61000
    mprotect(0x7fa49da62000, 8388608, PROT_READ|PROT_WRITE) = 0

    clone(child_stack=0x7fa49e260fb0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|C
LONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEA
RTID, parent_tid=[3737], tls=0x7fa49e261700, child_tidptr=0x7fa49e2619d0) = 3737
    futex(0x7fa49ea629d0, FUTEX_WAIT, 3736, NULL) = 0
    mmap(NULL, 200003584, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa484143000

```

```
mmap(NULL, 200003584, PROT_READ|PROT_WRITE,  
MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fa478286000
```

```
munmap(0x7fa484143000, 200003584)    = 0
```

```
munmap(0x7fa478286000, 200003584)    = 0
```

```
write(1, "Sorting completed in ", 21Sorting completed in ) = 21
```

```
write(1, "9491", 49491)              = 4
```

```
write(1, " ms\n", 4 ms
```

```
)              = 4
```

```
munmap(0x7fa49ea63000, 400003072)    = 0
```

```
exit_group(0)                        = ?
```

```
+++ exited with 0 +++
```

Вывод

В этой лабораторной работе реализован многопоточный алгоритм сортировки на основе Тим-сорт, который комбинирует вставочную сортировку для малых участков массива и сортировку слиянием для более крупных. Программа делит массив на части, обрабатываемые отдельными потоками, что ускоряет процесс сортировки на многопроцессорных системах. Каждый поток выполняет сортировку своей части массива, после чего результаты сливаются в общий отсортированный массив. Использование мьютексов обеспечивает синхронизацию потоков при объединении данных, а системные вызовы для работы с памятью и временем позволяют контролировать выполнение программы.